41039 Programming 1

Revision of Week 8: inheritance and interfaces



Dr. Xuan-Bach Le

le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 10 questions, about 35 minutes

#	Type	Topic
Q1	Fill in the blanks	`extends`, `super`, overriding
Q2	Multiple choice	Which line does not compile?
Q3	Multiple choice	Overloading or overriding?
Q4	Predict the output	Bikes and vehicles (Java)
Q5	Predict the output	Lamps and a fan (Python)
Q6	Find the bug	Cinema tickets (Java, 4 bugs)
Q7	Find the bug	Staff pay (Python, 3 bugs)
Q8	Write the code	A toll gate (Java)
Q9	Write the code	An account with a fee (Python)
Q10	Write the code	Greeters (Java)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. The answer appears on the **next click**.

Week 8 in five ideas (read this first)

1. **Is-a.** `class Dog extends Animal`: a Dog *is an* `Animal` and inherits its fields and methods. Python: `class Dog(Animal):`.
2. **`super`.** A constructor first runs `super(...)` to build the parent part (Python: `super().__init__(...)`). `super.m()` runs the **parent's** `m`.
3. **Override vs overload.** Same name **and** parameters: it *overrides* (`@Override` makes the compiler check). Different parameters: it *overloads*, a separate method.
4. **Declared type vs object.** `Animal a = new Dog();` The declared type decides what you may **call**; the object decides which override **runs**.
5. **Promises.** An `abstract` class cannot be `new`ed. An `interface` lists public methods; a class may `implement` many. Python: `ABC`, or *duck typing*.

```
interface Priced { int cost(); }
abstract class Item implements Priced {
    final String name;
    Item(String n) { name = n; }
}
class Book extends Item {
    Book(String n) { super(n); }
    @Override public int cost() { return 20; }
}
```

Q1 Fill in the blanks Inheritance words

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- A Java class names its superclass with **(1)** .
- **(2)** `(...)`, first in a constructor, runs the superclass constructor.
- Same name and parameters as a parent method: it **(3)** it. Different parameters: it **(4)** it.
- An **(5)** class cannot be created with `new`.
- Python **(6)** only asks if an object has the method.

Word bank (2 min, alone)

a) extends	**f)** overloads
b) implements	**g)** abstract
c) super	**h)** final
d) this	**i)** duck typing
e) overrides	**j)** static

Q1 Fill in the blanks Inheritance words

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- A Java class names its superclass with **(1)**.
- **(2)** `(...)`, first in a constructor, runs the superclass constructor.
- Same name and parameters as a parent method: it **(3)** it. Different parameters: it **(4)** it.
- An **(5)** class cannot be created with `new`.
- Python **(6)** only asks if an object has the method.

Answers

(1) a) extends **(2)** c) super
(3) e) overrides **(4)** f) overloads
(5) g) abstract **(6)** i) duck typing

Not used: b) implements (for interfaces), d) this (calls a constructor of the *same* class), h) final (a `final` class *can* be created, but not extended), j) static.

Q2 Multiple choice Which line does not compile?

```
interface Speaker { String speak(); }
abstract class Pet implements Speaker {
    protected final String name;
    Pet(String name) { this.name = name; }
}
class Cat extends Pet {
    Cat(String name) { super(name); }
    public String speak() { return name + ": meow"; }
    void purr() { System.out.println("purr"); }
}
```

Choose one (60 seconds)

A. `Speaker s = p;`
B. `String t = p.speak();`
C. `p.purr();`
D. `new Cat("Mo").purr();`

`main` is in another class, `Demo`, in the same file, and starts with:

```
Pet p = new Cat("Tom");
```

Each option is the next line of `main`.
Exactly **one** of them does not compile.

Q2 Multiple choice Which line does not compile?

```
interface Speaker { String speak(); }
abstract class Pet implements Speaker {
    protected final String name;
    Pet(String name) { this.name = name; }
}
class Cat extends Pet {
    Cat(String name) { super(name); }
    public String speak() { return name + ": meow"; }
    void purr() { System.out.println("purr"); }
}
```

Choose one (60 seconds)

A. `Speaker s = p;`

B. `String t = p.speak();`

✓ **C.** `p.purr();`

D. `new Cat("Mo").purr();`

`main` is in another class, `Demo`, in the same file, and starts with:

```
Pet p = new Cat("Tom");
```

Answer: C

The object is a `Cat`, but the compiler checks the **declared** type, `Pet`, which has no `purr`: `cannot find symbol`

A: `Pet implements Speaker`: a `Pet` *is a* `Speaker`.

B: `Pet` promises `speak()` through `Speaker`.

D: here the declared type is `Cat`.

Q3 Multiple choice Overloading or overriding?

```
class Animal {
    String sound() { return "..."; }
}
class Dog extends Animal {
    @Override String sound() { return "woof"; }
}
class Vet {
    static String check(Animal a) {
        return "animal " + a.sound();
    }
    static String check(Dog d) {
        return "dog " + d.sound();
    }
}
// in main, in another class Demo
Animal pet = new Dog();
System.out.println(Vet.check(pet));
```

What is printed? (90 seconds)

A. `dog woof`
B. `animal woof`
C. `animal ...`
D. `dog ...`

Hint: two different choices happen here. Which `check`? Then, inside it, which `sound`?

Q3 Multiple choice Overloading or overriding?

```
class Animal {
    String sound() { return "..."; }
}
class Dog extends Animal {
    @Override String sound() { return "woof"; }
}
class Vet {
    static String check(Animal a) {
        return "animal " + a.sound();
    }
    static String check(Dog d) {
        return "dog " + d.sound();
    }
}
// in main, in another class Demo
Animal pet = new Dog();
System.out.println(Vet.check(pet));
```

What is printed? (90 seconds)

A. `dog woof`
✓ **B.** `animal woof`
C. `animal ...`
D. `dog ...`

Answer: B

Which `check` (overload): the compiler picks it from `pet`'s declared type, `Animal`.
Which `sound` (override): picked as it runs, from the object, a `Dog`.
A, **D**: an overload does not follow the object. **C**: an override does.

Q4 Predict the output Bikes and vehicles

```
class Vehicle {
    protected int wheels;
    Vehicle(int wheels) {
        this.wheels = wheels;
        System.out.println("Vehicle " + wheels);
    }
    String info() { return wheels + " wheels"; }
}
class Bike extends Vehicle {
    Bike() {
        super(2);
        System.out.println("Bike");
    }
    @Override String info() {
        return "Bike, " + super.info();
    }
}
// in main, in another class Demo
Vehicle[] vs = { new Bike(), new Vehicle(4) };
for (Vehicle v : vs) System.out.println(v.info());
```

Predict (90 seconds, alone)

1. Write down **every** line that is printed, in order.

2. Delete the line `super(2);`. Does the program still compile? Why?

Q4 Predict the output Bikes and vehicles

```
class Vehicle {
    protected int wheels;
    Vehicle(int wheels) {
        this.wheels = wheels;
        System.out.println("Vehicle " + wheels);
    }
    String info() { return wheels + " wheels"; }
}
class Bike extends Vehicle {
    Bike() {
        super(2);
        System.out.println("Bike");
    }
    @Override String info() {
        return "Bike, " + super.info();
    }
}
// in main, in another class Demo
Vehicle[] vs = { new Bike(), new Vehicle(4) };
for (Vehicle v : vs) System.out.println(v.info());
```

Answers

1.
```
Vehicle 2
Bike
Vehicle 4
Bike, 2 wheels
4 wheels
```

`super(2)` runs **first**, so the parent part is built before `"Bike"` prints. Each object runs its own `info()`.

2. No. Java then calls `super()` with no arguments, and `Vehicle` has no such constructor: `constructor Vehicle in class Vehicle cannot be applied to given types`

Q5 Predict the output Lamps and a fan

```
class Lamp:
    def __init__(self, room):
        self.room = room
    def status(self):
        return self.room + " lamp"

class SmartLamp(Lamp):
    def status(self):
        return super().status() + " (smart)"

class Fan:
    def status(self):
        return "fan"

for d in [Lamp("hall"), SmartLamp("bed"), Fan()]:
    print(d.status())
print(isinstance(SmartLamp("x"), Lamp),
      isinstance(Fan(), Lamp))
```

`isinstance(x, C)`: is `x` a `C` (or a subclass)?

Predict (90 seconds, alone)

1. Write down the **four** lines that are printed.
2. Rename `Fan`'s method from `status` to `power`. What happens now?

Q5 Predict the output Lamps and a fan

```
class Lamp:
    def __init__(self, room):
        self.room = room
    def status(self):
        return self.room + " lamp"

class SmartLamp(Lamp):
    def status(self):
        return super().status() + " (smart)"

class Fan:
    def status(self):
        return "fan"

for d in [Lamp("hall"), SmartLamp("bed"), Fan()]:
    print(d.status())
print(isinstance(SmartLamp("x"), Lamp),
      isinstance(Fan(), Lamp))
```

`isinstance(x, C)`: is `x` a `C` (or a subclass)?

Answers

1.
```
hall lamp
bed lamp (smart)
fan
True False
```
`SmartLamp` uses `Lamp`'s `__init__`. A `Fan` is **not** a `Lamp`, but the loop only needs `status()`: **duck typing**.

2. Two lines print, then it stops: `AttributeError: 'Fan' object has no attribute 'status'`. Python checks only when the call runs.

Q6 Find the bug Cinema tickets (Java)

```java
interface Priced { int price(); }
public class Ticket implements Priced {
    protected final String film;
    Ticket(String film) { this.film = film; }
    int price() { return 15; }
    String label() {
        return film + " $" + price();
    }
}
class Student extends Ticket {
    Student(String film) { super("film"); }
    public int price() { return price() - 5; }
    String label(int seat) {
        return "Student " + super.label();
    }
}
```

✗ Find 4 bugs (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

What the classes should do

- A ticket costs 15; a student ticket 5 **less**
- `Student`'s `label()` starts `"Student "`

Test (`main` in class `Demo`)

```java
Ticket a = new Ticket("Dune");
Ticket b = new Student("Emma");
System.out.println(a.label());
System.out.println(b.label());
Priced p = b;
System.out.println(p.price());
```

Expected output

```
Dune $15
Student Emma $10
10
```

Q6 Find the bug Cinema tickets (Java)

```
1  interface Priced { int price(); }
2  public class Ticket implements Priced {
3      protected final String film;
4      Ticket(String film) { this.film = film; }
5      int price() { return 15; }
6      String label() {
7          return film + " $" + price();
8      }
9  }
10 class Student extends Ticket {
11     Student(String film) { super("film"); }
12     public int price() { return price() - 5; }
13     String label(int seat) {
14         return "Student " + super.label();
15     }
16 }
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

✓ The 4 fixes

- **Line 5** (compile error): interface methods are `public`, and an implementation may not be less visible: `attempting to assign weaker access privileges; was public`. Add `public`.
- **Line 11**: passes the text `"film"`, not the parameter. Use `super(film)`.
- **Line 12**: `price()` calls this same method again, forever (`StackOverflowError`). Use `super.price() - 5`.
- **Line 13**: different parameters, so it *overloads*: `b.label()` still runs `Ticket`'s. Write `@Override String label()`.

Fix only line 5: `Dune $15`, then a crash.

Q7 Find the bug Staff pay (Python)

```python
1  class Employee:
2      def __init__(self, name, salary):
3          self.name = name
4          self.salary = salary
5      def pay(self):
6          return self.salary
7      def __str__(self):
8          return self.name + ": " + str(self.pay())
9
10 class Manager(Employee):
11     def __init__(self, name, salary, bonus):
12         self.bonus = bonus
13     def Pay(self):
14         return super().pay + self.bonus
15
16 staff = [Employee("An", 100),
17          Manager("Binh", 100, 50)]
18 for e in staff:
19     print(e)
```

✗ **Find 3 bugs** in lines 10–14 (3 min).
Give each line number and its fix.

What the classes should do

- A `Manager` is an `Employee` with a bonus
- A manager's `pay()` is salary + bonus
- Printing anyone shows "<name>: <pay>"

Expected output

```
An: 100
Binh: 150
```

✗ Actual output (crashes at line 8)

```
An: 100
AttributeError: 'Manager' object has no
  attribute 'name'
```

Q7 Find the bug Staff pay (Python)

```
1  class Employee:
2      def __init__(self, name, salary):
3          self.name = name
4          self.salary = salary
5      def pay(self):
6          return self.salary
7      def __str__(self):
8          return self.name + ": " + str(self.pay())
9
10 class Manager(Employee):
11     def __init__(self, name, salary, bonus):
12         self.bonus = bonus
13     def Pay(self):
14         return super().pay + self.bonus
15
16 staff = [Employee("An", 100),
17          Manager("Binh", 100, 50)]
18 for e in staff:
19     print(e)
```

✗ **Find 3 bugs** in lines 10–14 (3 min).
Give each line number and its fix.

✓ The 3 fixes

- **Line 12**: `Manager`'s `__init__` replaces `Employee`'s, so `name` and `salary` are never set. Add `super().__init__(name, salary)` first.
- **Line 13**: `Pay` is not `pay` (case matters), so nothing is overridden: it would print `Binh: 100`. Rename it `pay`.
- **Line 14**: `super().pay` without `()` is the method, not its result: `TypeError` (method + int). Write `super().pay() + self.bonus`.

Q8 Write the code A toll gate (Java)

Finish `Truck` (alone, 4 min). `toll()`: 4 per axle, but at least 10.

- Constructor: pass `plate` up to `Vehicle` (it is `private` there); keep `axles` in a new field.

Skeleton

```
abstract class Vehicle {
    private final String plate;
    Vehicle(String plate) { this.plate = plate; }
    abstract int toll();
    public String toString() {
        return plate + " pays " + toll();
    }
}
class Car extends Vehicle {
    Car(String plate) { super(plate); }
    int toll() { return 5; }
}
class Truck extends Vehicle {
    Truck(String plate, int axles) {
        super("?");  // TODO
    }
    int toll() { return 0; }  // TODO
}
```

Test (in `main`)

```
Vehicle[] queue = { new Car("CAR-1"),
    new Truck("TRK-2", 2), new Truck("TRK-7", 3) };
int total = 0;
for (Vehicle v : queue) {
    System.out.println(v);
    total += v.toll();
}
System.out.println("Total " + total);
```

Must print

```
CAR-1 pays 5
TRK-2 pays 10
TRK-7 pays 12
Total 27
```

Q8 Write the code A toll gate (Java)

Finish `Truck` (alone, 4 min). `toll()`: 4 per axle, but at least 10.

- Constructor: pass `plate` up to `Vehicle` (it is `private` there); keep `axles` in a new field.

Skeleton

```
abstract class Vehicle {
    private final String plate;
    Vehicle(String plate) { this.plate = plate; }
    abstract int toll();
    public String toString() {
        return plate + " pays " + toll();
    }
}
class Car extends Vehicle {
    Car(String plate) { super(plate); }
    int toll() { return 5; }
}
class Truck extends Vehicle {
    Truck(String plate, int axles) {
        super("?");  // TODO
    }
    int toll() { return 0; }  // TODO
}
```

✓ One answer

```
class Truck extends Vehicle {
    private int axles;
    Truck(String plate, int axles) {
        super(plate);
        this.axles = axles;
    }
    int toll() { return Math.max(10, 4 * axles); }
}
```

`super(plate)` comes first. The loop never asks which kind of vehicle it has: each object runs its own `toll()`.

Q9 Write the code An account with a fee (Python)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `__init__`: keep the `fee`. `withdraw(amount)`: take `amount + fee` by `Account`'s rule. **Reuse** its `withdraw` with `super()`; do not copy it.

Skeleton

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
    def withdraw(self, amount):
        if amount > self.balance:
            return False
        self.balance -= amount
        return True

class FeeAccount(Account):
    def __init__(self, owner, balance, fee):
        super().__init__(owner, balance)
        # TODO: keep the fee
    def withdraw(self, amount):
        # TODO
        return False
```

Test

```
a = Account("An", 10)
f = FeeAccount("Binh", 10, 2)
for acc in [a, f]:
    print(acc.owner, acc.withdraw(8), acc.balance)
print(f.withdraw(1), f.balance)
```

Must print

```
An True 2
Binh True 0
False 0
```

Q9 Write the code An account with a fee (Python)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `__init__`: keep the `fee`. `withdraw(amount)`: take `amount + fee` by `Account`'s rule. **Reuse** its `withdraw` with `super()`; do not copy it.

Skeleton

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
    def withdraw(self, amount):
        if amount > self.balance:
            return False
        self.balance -= amount
        return True

class FeeAccount(Account):
    def __init__(self, owner, balance, fee):
        super().__init__(owner, balance)
        # TODO: keep the fee
    def withdraw(self, amount):
        # TODO
        return False
```

✓ One answer

```
class FeeAccount(Account):
    def __init__(self, owner, balance, fee):
        super().__init__(owner, balance)
        self.fee = fee
    def withdraw(self, amount):
        total = amount + self.fee
        return super().withdraw(total)
```

Binh: $8 + 2 = 10$, so 0 is left. Then $1 + 2 = 3 > 0$: refused, nothing changes. The rule lives in one place, `Account`.

Q10 Write the code Greeters (Java)

Finish the `TODO`s (alone, 4 min). `Vietnamese` says `"Xin chao, <name>"`.

- Loud **has a** `Greeter`: return its greeting in capitals, plus `"!"` (use `s.toUpperCase()`).

Skeleton

```
interface Greeter { String greet(String name); }
class English implements Greeter {
    public String greet(String name) {
        return "Hello, " + name;
    }
}
class Vietnamese implements Greeter {
    public String greet(String name) {
        return "";  // TODO
    }
}
class Loud implements Greeter {
    private final Greeter inner;
    Loud(Greeter inner) { this.inner = inner; }
    public String greet(String name) {
        return "";  // TODO
    }
}
```

Test (in `main`)

```
Greeter[] gs = { new English(), new Vietnamese(),
    new Loud(new English()),
    new Loud(new Vietnamese()) };
for (Greeter g : gs)
    System.out.println(g.greet("Lan"));
```

Must print

```
Hello, Lan
Xin chao, Lan
HELLO, LAN!
XIN CHAO, LAN!
```

Q10 Write the code Greeters (Java)

Finish the `TODO`s (alone, 4 min). `Vietnamese` says `"Xin chao, <name>"`.

- Loud **has a** `Greeter`: return its greeting in capitals, plus `"!"` (use `s.toUpperCase()`).

Skeleton

```
interface Greeter { String greet(String name); }
class English implements Greeter {
    public String greet(String name) {
        return "Hello, " + name;
    }
}
class Vietnamese implements Greeter {
    public String greet(String name) {
        return "";  // TODO
    }
}
class Loud implements Greeter {
    private final Greeter inner;
    Loud(Greeter inner) { this.inner = inner; }
    public String greet(String name) {
        return "";  // TODO
    }
}
```

✓ One answer

```
class Vietnamese implements Greeter {
    public String greet(String name) {
        return "Xin chao, " + name;
    }
}
class Loud implements Greeter {
    private final Greeter inner;
    Loud(Greeter inner) { this.inner = inner; }
    public String greet(String name) {
        String g = inner.greet(name);
        return g.toUpperCase() + "!";
    }
}
```

Loud never asks which class `inner` is: any `Greeter` works.

Looking ahead: when the input is not a number

Count the axles in some lines of text

```
String[] lines = {"3", "two", "5"};
int axles = 0;
for (String s : lines)
    axles += Integer.parseInt(s);
System.out.println(axles);
```

Think (1 minute)

What happens at `"two"`? How could the program **skip** a bad line and still print a total?

Looking ahead: when the input is not a number

Count the axles in some lines of text

```
String[] lines = {"3", "two", "5"};
int axles = 0;
for (String s : lines)
    axles += Integer.parseInt(s);
System.out.println(axles);
```

Think (1 minute)

What happens at `"two"`? How could the program **skip** a bad line and still print a total?

It stops, and prints no total:

```
java.lang.NumberFormatException:
  For input string: "two"
```

Instead, **catch** the failure:

```
for (String s : lines) {
    try {
        axles += Integer.parseInt(s);
    } catch (NumberFormatException e) {
        System.out.println("skip " + s);
    }
}
```

Now it prints `skip two`, then 8. Exceptions are **objects** in a class hierarchy, so inheritance is back. Python: `except ValueError`. Today: **exceptions** and **reading text**.

You should now be able to...

Inheritance

- ✓ write `extends` and `super(...)` so the parent part is built first (Q1, Q4, Q8)
- ✓ reuse the parent's version with `super.m()` / `super().m()` (Q4, Q5, Q7, Q9)
- ✓ tell overriding from overloading, and use `@Override` (Q1, Q3, Q6, Q7)
- ✓ predict which method runs: declared type vs object (Q2, Q3, Q4)

Abstract classes and interfaces

- ✓ use an `abstract` class: no `new`, subclasses fill in (Q1, Q8)
- ✓ implement an interface with `public` methods (Q2, Q6, Q10)
- ✓ give an object a collaborator through an interface (Q10)
- ✓ explain Python duck typing, and when it fails (Q5)

Next: Week 9 – Exceptions and reading text

`try`/`catch` and `except`, throwing and passing on exceptions, `finally`, and reading tokens and lines until the input ends.